实验二　存储层次设计空间探索

> 4 学时　|　贯穿负载：qnn（int8 量化神经网络推理）　前置：实验一。**如果你还不清楚 `l1d_misses` 和 `l1d_mshr_misses` 的区别，先回去看实验一的任务 3，本次实验从头到尾都要用它。**

本次实验要你做出的决策

> **你手上有一份固定的硅片面积预算。这些面积应该花在：L1D 更大？相联度更高？块更大？还是给 L2？**

这一次没有"标准配置"。你要交出的是一份**配置提案 + 数据论证**，而且必须回答"为什么不是另外那三个选项"。

一、补课：Cache 到底在解决什么问题

> 学过《计算机体系结构》的同学可以跳到 §1.5，但 §1.4 的 MSHR 建议看一眼。

1.1 存储墙

CPU 每个周期能做几条指令，而一次 DRAM 访问要 **100~300 个周期**。如果每条访存指令都去 DRAM，处理器 99% 的时间在等。

Cache 是一块小而快的存储，放在 CPU 和 DRAM 之间。它能起作用，全靠程序的**局部性**：

- **时间局部性**：刚用过的数据，很可能马上还要用（循环变量、栈上的局部变量）
- **空间局部性**：用了 `a[i]`，很可能马上要用 `a[i+1]`（数组遍历）

Cache 不按字节存，而是按**块（cache line/block）**存，一块通常 64 字节。一次失效就把整块搬进来——**这就是在赌空间局部性。**

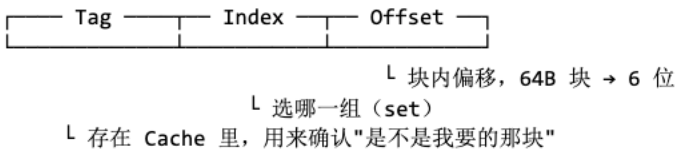
1.2 三种失效（3C 模型）

类型	英文	什么时候发生	怎么消除
强制失效	compulsory	这个块**第一次**被访问	只能靠**更大的块**或预取
容量失效	capacity	Cache 装不下工作集，被挤出去了	只能靠**更大的容量**
冲突失效	conflict	容量够，但几个块争同一组	靠**更高的相联度**

> 这个模型是本次实验的分析框架。你测出失效变多或变少时，**第一件事是判断变的是哪一个 C**——因为三个 C 的对策完全不同，判错了就会把面积花在错误的地方。

1.3 组相联怎么工作

一个地址被拆成三段：



- **组数** = 容量 ÷ (块大小 × 相联度)
- **直接映射（1 路）**：每个地址只能放在唯一确定的一个位置。查找最快，但两个地址 只要 Index 相同就会互相踢，哪怕 Cache 空着一大半。
- **N 路组相联**：每组有 N 个位置，新块可以挑一个放。冲突少了， 但要同时比较 N 个 Tag，**命中延迟变长、功耗变高。**
- **全相联**：任意位置都能放，冲突失效为 0，但硬件代价大到只有 TLB 这种小结构才用。

1.4 ★ 非阻塞 Cache 与 MSHR（实验一已经讲过，这里定型）

现代乱序处理器的 Cache 在失效之后**不会停下来**，而是继续接受新的访问。 支撑这件事的硬件叫 **MSHR（Miss Status Holding Register，失效状态保持寄存器）。**

一次失效发生时：

1. Cache 分配一个 MSHR，记下"我在等哪个块、回来后给谁"，然后向下一级发请求；
2. 在数据回来之前，如果又有访问命中**同一个块**， 不再发新请求，而是挂到那个 MSHR 上一起等——这叫 **MSHR hit**（次级失效）；
3. MSHR 用完了，Cache 才真正阻塞。

所以：

$$\underbrace{\text{overallMisses}}_{\text{失效次数}} = \underbrace{\text{overallMshrHits}}_{\text{被合并的}} + \underbrace{\text{overallMshrMisses}}_{\text{真正下到 L2 的流量}}$$

> **本次实验的第一条规则： 判断"访存压力大不大"，看 `l1d_mshr_mpki`，不要看 `l1d_miss_rate`。** 前者是真实流量，后者被乱序核的重复访问严重放大。 你会在任务 4 里看到：同一个程序、同样的数据，只改一个硬件参数， "失效率"能从 12.2% 变成 0.78%——**失效率是微体系结构的产物，不是程序的属性。**

我们还用 **MLP（Memory Level Parallelism）** 表示平均有多少个失效同时在飞：

$$\text{MLP} = \frac{\text{overallMisses}}{\text{overallMshrMisses}}$$

1.5 ★★ 为什么"越大越好"是错的

初学者最容易犯的错：把 Cache 越配越大，看到失效率下降就宣布"改进了"。

真实的芯片里，Cache 越大，访问它就越慢。 原因是物理的： 容量大 → 阵列的字线/位线更长 → 寄生电容更大 → 充放电更慢； 同时译码器层数增加、Tag 比较的扇出变大。

工程上常用的经验规律是**容量每翻一番，命中延迟约 +1 个周期。** 本课程的 `framework/cssys_se.py` 里的 `scaled_latency()` 就是这个模型：

```
# 以 32kB / 3 周期为基准，每翻一番 +1
l1_latency = base + log2(size / 32kB)
```

于是就有了一个**真正的权衡**：

$$\text{AMAT} = \underbrace{\text{命中延迟}}_{\text{随容量}\uparrow} + \underbrace{\text{失效率}}_{\text{随容量}\downarrow} \times \text{失效代价}$$

（AMAT = Average Memory Access Time，平均访存时间）

两项一升一降，最优点在中间某处——**这就是本次实验要你找的东西。**

> 我们提供了一个开关 `--auto-latency off` 可以关掉这个模型。 **任务 1 会让你把它打开和关掉各跑一遍**，你会看到关掉之后 曲线变成单调的"越大越好"——那是一个**因为模型缺失而产生的假结论。** 这是本课程反复要讲的一件事：**你的结论受你的模型限制。**

1.6 qnn 的工作集有多大（动手前先算）

打开 `bench/src/qnn.c`，把数组尺寸抄出来：

数组	尺寸	字节数
`w1[128][784]`	128 × 784 × 1B	100,352
`w2[10][128]`	10 × 128 × 1B	1,280
`b1[128]`	128 × 4B	512
`xbuf[784]`	784 × 1B	784
其余		~208
合计		**≈ 100.7 KB**

记住这个数：≈ 100 KB，其中 98 KB 是 `w1`。 本次实验几乎每一条曲线的形状都由它决定。

由此你可以**在跑之前就预测**每个样本要产生多少次强制失效：

$$\text{每样本失效数} \approx \frac{100.7\ \text{KB}}{\text{块大小}}$$

任务 3 会让你验证这个预测。**能预测的实验才叫实验，否则只是在采集数据。**

二、实验环境

和实验一相同。本次实验会用到的新参数：

```
--l1d-size 32kB        # L1 数据 Cache 容量
--l1d-assoc 2          # 相联度
--l1d-repl lru         # 替换策略：lru / random / fifo / rrip / ...
--l1d-mshrs 4          # MSHR 个数
--cacheline 64         # 块大小（同时作用于 L1I/L1D/L2）
--l2-size 256kB
--auto-latency on|off  # 是否让命中延迟随容量增长
```

默认基线（后面所有"相对基线"都指它）：

```
--cpu o3 --l1d-size 32kB --l1d-assoc 2 --l1d-mshrs 4 --cacheline 64 --l2-size 256kB
```

> ⚠️ 继续用你自己的学号后四位作 seed。 ⚠️ **实验一的规则继续有效**：本环境噪声地板约 0.01%， **小于 0.1% 的差异一律当作"没有差异"**，不用解释。

三、动手任务

任务 1　容量：找到那个拐点（40 分钟）

```
python3 framework/sweep.py --name lab2_size \
    --bench qnn --bench-args "8 <学号后四位>" \
    --base "--cpu o3" \
    --vary l1d-size=8kB,16kB,32kB,64kB,128kB,256kB --jobs 6
python3 framework/report.py results/lab2_size.csv \
    cycles ipc l1d_miss_rate l1d_mpki l1d_mshr_mpki l1d_mlp
```

填表并画图（横轴容量取对数刻度，纵轴画**周期数**）：

容量	周期数	IPC	l1d_miss_rate	l1d_mpki	l1d_mshr_mpki
8kB					
…					

要回答：

1. 周期数最少的是哪个容量？它比基线（32kB）快多少？
2. **8kB → 64kB 这一段，周期数是往上走的**（Cache 变大反而变慢）。 看 l1d_mshr_mpki 这一列，这一段里它变了吗？ 由此说明：这一段的性能变化，收益和代价各是什么？

3. 128kB 处发生了什么？用 §1.6 算出的工作集大小解释这个突变。
4. 256kB 又变慢了。为什么？

（b）关掉延迟模型，看看会得出什么假结论

```
python3 framework/sweep.py --name lab2_size_nolat \
    --bench qnn --bench-args "8 <学号后四位>" \
    --base "--cpu o3 --auto-latency off" \
    --vary l1d-size=8kB,16kB,32kB,64kB,128kB,256kB --jobs 6
```

把两条曲线画在**同一张图**上。

要回答： 5. 关掉延迟模型后曲线是什么形状？如果你只有这条曲线，你会选多大的 Cache？ 6. 这两条曲线里，哪一条更接近真实芯片？说出你的理由。 7. **推广一下**：模拟器没有建模的东西，会以什么方式出现在你的结论里？

任务 2　相联度：一个反直觉的结果（50 分钟）★★核心任务

课本告诉你"相联度越高，冲突失效越少"。我们来验证一下。

```
python3 framework/sweep.py --name lab2_assoc \
    --bench qnn --bench-args "8 <学号后四位>" \
    --base "--cpu o3" \
    --vary l1d-size=32kB,64kB,128kB --vary l1d-assoc=1,2,4,8 --jobs 12
python3 framework/report.py results/lab2_assoc.csv \
    cycles ipc l1d_miss_rate l1d_mshr_mpki
```

填表（重点看 **64kB** 那一行）：

容量	1 路	2 路	4 路	8 路
32kB `mshr_mpki`				
64kB `mshr_mpki`				
128kB `mshr_mpki`				

要回答：

1. 64kB 时，**1 路（直接映射）比 2/4/8 路的失效更少还是更多**？ 这符合课本说法吗？
2. 3C 模型说"提高相联度只会减少冲突失效，不会增加任何失效"。 你的数据和这句话矛盾吗？**那么这句话隐含了什么前提？**

（b）换一个替换策略

上面的实验一直用 LRU（最近最少使用）。换成随机替换再跑一遍：

```
python3 framework/sweep.py --name lab2_repl \
    --bench qnn --bench-args "8 <学号后四位>" \
    --base "--cpu o3 --l1d-size 64kB" \
    --vary l1d-assoc=1,2,4,8 --vary l1d-repl=lru,random --jobs 8
```

相联度	LRU `mshr_mpki`	LRU 周期	Random `mshr_mpki`	Random 周期
1				
2				
4				
8				

要回答：

3. **先做一个自检**：1 路时 LRU 和 Random 的结果应该完全相同。是吗？ 为什么必须相同？ （如果不同，说明你哪里配错了，先排查再往下做）
4. Random 下，相联度提高时失效是单调下降的吗？LRU 呢？
5. **解释 LRU 为什么会输。** 提示：
   - qnn 的内层循环在干什么？`dot_i8(w1[j], xbuf, 784)` 每次调用扫过多长的数据？
   - 每个样本要把 `w1` 整个（98 KB）**从头到尾扫一遍**，然后**再扫一遍。**
   - 假设你有一个 64 KB 的 Cache，要反复循环访问一个 98 KB 的数组。 当你访问到数组末尾、准备回到开头时，LRU 保留的是哪一部分数据？ 你**马上要用**的是哪一部分？
6. 由此给出一条设计结论：**什么样的访问模式下不应该用 LRU?** 查一下真实处理器（如 Intel、ARM）的 L1 用的是什么替换策略， 为什么不是教科书上的标准 LRU?

> 这一题是本次实验的重点。它要打掉的是"课本结论可以无条件套用"这个习惯。 3C 模型没有错，错的是把它当成一个与替换策略无关的定律。

任务 3　块大小：三股力量在拔河（40 分钟）★

```
python3 framework/sweep.py --name lab2_line \
    --bench qnn --bench-args "8 <学号后四位>" \
    --base "--cpu o3" \
    --vary cacheline=16,32,64,128,256 --jobs 5
```

(a) 先预测，再测量

用 §1.6 的公式，**在看结果之前**填出预测列：

块大小	预测每样本失效数 (100.7KB ÷ 块大小)	预测 `mshr_mpki` (×8 样本 ÷ 5.8M 指令 ×1000)	实测 `mshr_mpki`	误差
16B				

块大小	预测每样本失效数 (100.7KB ÷ 块大小)	预测 mshr_mpki (×8 样本 ÷ 5.8M 指令 ×1000)	实测 mshr_mpki	误差
32B				
64B				
128B				
256B				

要回答：

1. 你的预测准吗？误差多少？
2. mshr_mpki 在块大小翻倍时是什么规律？为什么是这个规律？ （提示：搬运的总字节数变了吗？）

（b）性能为什么不跟着走

块大小	周期数	IPC	l1d_mshr_mpki	l1d_mlp
16B				

…

要回答： 3. 失效数从 16B 到 256B 降了 16 倍，周期数降了几倍？为什么不成比例？ 4. **16B 的性能崩塌得特别厉害**（比 32B 慢将近 4 倍）。 先提出一个假设，然后**设计实验验证它。**

一个很自然的假设是"MSHR 不够用了"。验证它：

```
python3 framework/sweep.py --name lab2_line_mshr \
    --bench qnn --bench-args "8 <学号后四位>" \
    --base "--cpu o3" \
    --vary cacheline=16,32,64 --vary l1d-mshrs=4,16,64 --jobs 9
```

再查一个直接证据：

```
grep 'dcache.blockedCycles::no_mshrs' out/sweeps/lab2_line_mshr/*/stats.txt
```

这个假设成立吗？ 如果不成立，说明它错在哪。

5. 假设被否定之后，去查取指侧的统计：

```
grep -E 'fetch.cacheLines|icacheStallCycles|cpu.numCycles ' \
     out/sweeps/lab2_line/p000_cacheline-16/stats.txt
```

icacheStallCycles 占总周期的百分之几？和 64B 的比一比。

> **背景**：取指单元一次最多只能从**一个 Cache 块**里取指令 （framework/cssys_se.py 里 `fetchBufferSize = min(64, cacheline)`， 这不是我们的限制，是物理约束——你不可能一次读出还没搬进来的块）。 块变小，取指单元就要更频繁地重新访问 I-Cache。

用这个机制解释 16B 的崩塌。**注意这是一个"数据 Cache 参数拖垮了取指"的例子。**

6. 在 qnn 上，块越大越好，一路好到 256B。**那块大小应该无限增大吗?**

   先说清楚 qnn 为什么这么偏爱大块：dot_i8 是**连续**扫描 w1[j] 的 784 个字节， 搬进来的每一个字节都会被用到——**空间局部性 100%。**

   现在换一个**没有**空间局部性的负载来测：

   ```
   python3 framework/sweep.py --name lab2_line_pchase \
       --bench pchase --bench-args "256 200000 <学号后四位>" \
       --base "--cpu o3" --vary l1d-size=32kB,128kB --vary cacheline=64,256 --jot
   ```

   要回答：

   - 在 pchase 上，256B 块比 64B 块更快还是更慢？**在两个容量下答案一样吗?**
   - 用"搬进来的字节里有多少被用到"解释 qnn 和 pchase 的差别。
   - 顺便看一眼 pchase 的 l1d_mlp，和 qnn 的比一比。差这么多是为什么？ （提示：指针追逐的下一个地址是从哪来的？能提前算出来吗？）
   - 除了浪费带宽，块变大在**多核**系统里还有一个著名的代价，叫什么？ （关键词：**假共享 false sharing**）

任务 4　MSHR：失效率是谁的属性（25 分钟）

```
python3 framework/sweep.py --name lab2_mshr \
    --bench qnn --bench-args "8 <学号后四位>" \
    --base "--cpu o3" \
    --vary l1d-mshrs=1,2,4,8,16 --jobs 5
python3 framework/report.py results/lab2_mshr.csv \
    cycles ipc l1d_miss_rate l1d_mshr_mpki l1d_mlp
```

MSHR 数	周期数	l1d_miss_rate	l1d_mshr_mpki	l1d_mlp
1				
…				

要回答：

1. **l1d_miss_rate 在 MSHR=1 和 MSHR=2 之间发生了什么?** 变化有多大？而 l1d_mshr_mpki 变了吗?
2. 程序没变、数据没变、Cache 容量没变，**只改了一个队列的深度， "失效率"就变了十几倍。** 这说明"失效率"到底是什么的属性?
3. MSHR 从 4 增加到 16，性能有提升吗？为什么？ （提示：看 l1d_mlp 稳定在多少）

4. 由此判断：对 qnn 这个负载，**再加 MSHR 是不是一笔划算的投资**？什么样的负载会需要更多 MSHR？

任务 5　L2：钱该花在哪一级（25 分钟）

```
python3 framework/sweep.py --name lab2_l2 \
    --bench qnn --bench-args "8 <学号后四位>" \
    --base "--cpu o3" \
    --vary l2-size=128kB,256kB,512kB,1MB,2MB --jobs 5
python3 framework/report.py results/lab2_l2.csv \
    cycles ipc l2_miss_rate l2_accesses dram_bytes
```

要回答：

1. L2 从 128kB 加到 2MB，性能是变好还是变坏？变化幅度多少？
2. 看 `dram_bytes` 那一列，绝对数值是多大？相对于 ROI 的总访存量（约 160 万次），DRAM 流量占多少？
3. 由此判断：**对 qnn 这个负载，L2 起到作用了吗？**如果你是芯片架构师，你会把 L2 的面积挪去干什么？
4. 什么样的负载会让 L2 变得重要？（结合工作集大小回答）

任务 6　综合：拿出你的配置提案（40 分钟）★交付物

现在做决策。规则：

- 你必须在 `l1d-size`、`l1d-assoc`、`cacheline`、`l2-size` 四个维度上各选一个值；
- 你的提案必须**同时给出**：配置、相对基线的加速比、以及**为什么不选另一个值**；
- **必须实测**你提案的那个配置点，不能只靠前面单维扫描的结果外推。

```
# 换成你自己的选择
python3 framework/sweep.py --name lab2_final \
    --bench qnn --bench-args "8 <学号后四位>" \
    --base "--cpu o3 --l1d-size ??? --l1d-assoc ??? --cacheline ??? --l2-size ???"
    --vary l1d-repl=lru,random --jobs 2
```

要回答：

1. 你的提案配置是什么？相对基线加速多少？
2. **实测值和你从单维扫描外推的预测值一致吗？**如果不一致，说明这几个参数之间存在**交互作用**——举出一对相互影响的参数。
3. 你的提案对**其他负载**还成立吗？用 `stream` 或 `pchase` 验证一个：

```
python3 framework/sweep.py --name lab2_final_x \
    --bench pchase --bench-args "256 200000 <学号后四位>" \
    --base "<你的配置>" --vary l1d-size=32kB,<你选的容量> --jobs 2
```

如果结论翻转了，说明什么？

> **本任务的评分重点不是"你选得对不对"，而是"你的论证站不站得住"。** 一个选了次优配置但把权衡讲清楚的提案，分数高于一个碰巧选对但说不出理由的。

四、思考题（写进报告）

1. 任务 1 里，8kB 的 Cache 比 64kB 的**更快**。 如果你只报告"失效率"这一个指标，你会得出什么结论？ 这说明**用单一指标评价设计**有什么危险？

2. 本次实验所有的"容量→延迟"关系都来自 `scaled_latency()` 这个**经验模型**， 其中基准延迟（32kB 对应几个周期）是一个可调参数。 **你在任务 1 得到的最优容量，有多依赖这个参数？**

   ```
   python3 framework/sweep.py --name lab2_latsens \
       --bench qnn --bench-args "8 <学号后四位>" --base "--cpu o3" \
       --vary l1d-size=8kB,32kB,128kB,256kB --vary l1-latency=2,3,5 --jobs 12
   ```

   分别回答：**最优点的位置**变了吗？**最优点的领先幅度**变了吗？ 这两个问题的答案不一样，而这个差别正是"结论稳健性"的含义。

3. 任务 2 发现 LRU 在特定访问模式下不如随机替换。 工业界为此提出过许多"抗颠簸"的替换策略。

   （a）查阅资料，说明 **RRIP（Re-Reference Interval Prediction）** 声称要解决 什么问题，它的基本思想是什么。

   （b）**实测**一下，看它在我们的负载上是否兑现了承诺：

   ```
   python3 framework/sweep.py --name lab2_replfam \
       --bench qnn --bench-args "8 <学号后四位>" \
       --base "--cpu o3 --l1d-size 64kB --l1d-assoc 4" \
       --vary l1d-repl=lru,random,fifo,nru,rrip,brrip --jobs 6
   ```

   （c）结果符合你在 (a) 里的预期吗？**如果不符合，列出至少两种可能的原因** （提示：论文里的算法、模拟器里的实现、以及实现的默认参数，是三样不同的东西）。

   > 这一题**没有标准答案**，它考的是你面对"理论与实测不符"时的反应。 直接把实测结果写成"因此 RRIP 没用"是不合格的； 直接无视实测结果、照抄论文结论也是不合格的。

4. qnn 的 ROI 从**权重初始化之后**开始计时。 这意味着在 128kB 配置下，ROI 一开始 Cache 里就已经装好了 `w1`； 而在 64kB 配置下装不下。 **这对"128kB 更好"这个结论是不是不公平？** 设计一个实验来判断这个效应有多大。 （这是实验一思考题 4 的延续。真实部署中权重确实是加载一次反复使用的 —— 所以这里到底算不算"不公平"，本身就是一个值得讨论的建模问题。）

5. 本次实验我们一次只改一个参数。 如果四个参数各有 5 个取值，穷举要 625 个配置点。 按每点 20 秒、8 核并行算，要跑多久？ 工业界如何在不穷举的情况下探索这么大的设计空间?

五、提交要求

- 任务 1~6 的全部表格
- **至少三张图**：容量曲线（含 `auto-latency` 开/关两条）、相联度×替换策略、块大小
- 每个"要回答"的回答，结论必须指向你自己的数据
- **一份配置提案**（任务 6），不超过 400 字，包含四个维度的取值、 加速比、以及每个维度"为什么不选别的"

同时提交 `results/` 下所有 CSV。

> 评分权重：机制分析 45%，方法学 15%，配置提案 15%，数据完整性 15%，报告规范 10%。 **任务 2 和任务 3 的分析占了机制分析的一多半。**

附录：常见问题

Q: `l1d_miss_rate` 有 12%，是不是 Cache 配置错了？ 不是。看 `l1d_mshr_mpki`。见 §1.4。这是本次实验最常见的误解。

Q: 设了 `--cacheline 16`，gem5 报 fetch buffer 相关的错 不会。`cssys_se.py` 已经自动把取指缓冲限制到 `min(64, cacheline)`。 但这正是任务 3 问题 5 要研究的效应——**别把它当成 bug 绕过去。**

Q: `--l1d-size 96kB` 跑不起来 容量必须让"组数 = 容量 ÷ (块大小 × 相联度)"是 2 的幂。用 2 的幂次容量。

Q: 两个同学的拐点不在同一容量 不该发生。工作集大小由数组尺寸决定，与 seed 无关，拐点应该都在 128kB。 如果不同，检查是不是改了 `--cacheline` 或 `--l1d-assoc`。

Q: 我的提案配置实测比预测差很多 这是**正常且值得写进报告的。**参数之间有交互作用，单维扫描的结论不能简单叠加。 任务 6 问题 2 就是要你发现这一点。